Neuromorphic engineering I

Lab 2: Subthreshold Behavior of Transistors

Reminder: Did you `git pull` the exercises before starting this notebook?

This is reference solution to Lab2

Team member 1: Tobi Delbruck

Team member 2:

Date: 12 Oct 23

CoACH chip number: 29

In this lab exercise we will be investigating the subthreshold (weak inversion) behavior of isolated *p*-- and *n*--channel MOSFETs with $|V_{ds}| > 4U_T$, i.e. when the drain is saturated. Specifically, we will

- measure the currents through the transistors as a function of their gate voltage with a big V_{ds}
- compare the characteristics of p and n-fet devices w.r.t. the drain current vs gate voltage transconductance (κ) and compare the off current (I_0)

1. Prelab

Make sure you have studied the lecture material before attempting this prelab. The questions will also make much more sense if you read through the entire lab handout first. *You are required to complete this prelab before you can begin taking data.*

n- and p-fets, in an *n* well Process

A vertical section through the silicon with both n and p-fet transistors is shown in Figure 1. The class chip has a p-type substrate (like almost all chips nowadays) and both p- and n-wells.
The p-wells (not shown in the figure) are shorted to the p-substrate because the doping is of the same type.

Because we are grounding the substrate and we are connecting *n*--well to the power supply, V_{dd} is positive. This positive voltage reverse biases the junction between the *n*--wells (which are tied to V_{dd}) and the substrate (which is tied to gnd).

For this process, V_{dd}=1.8 V.

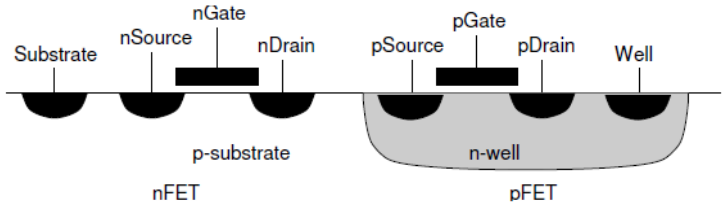
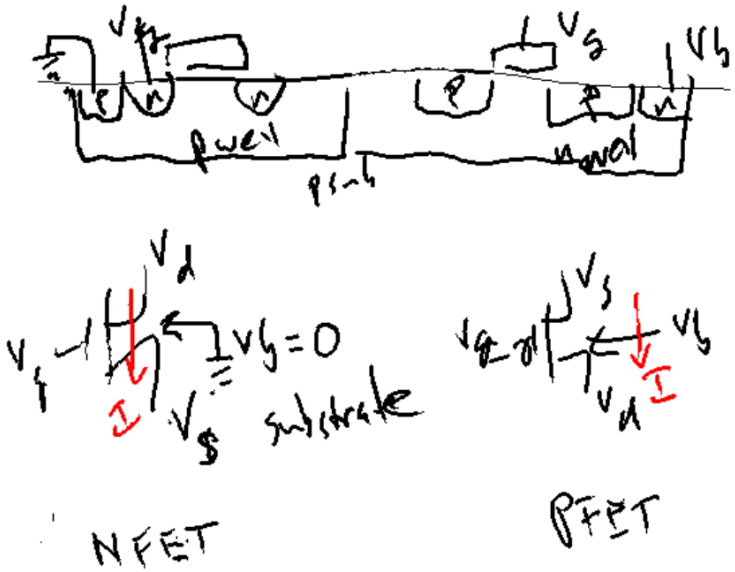


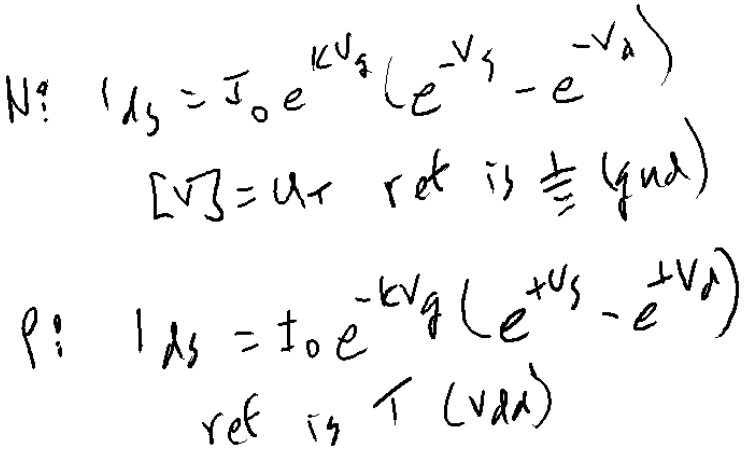
Figure 1: a cross section through p-substrate chip.

For the following questions assume an *n*--well process -- unless stated otherwise.

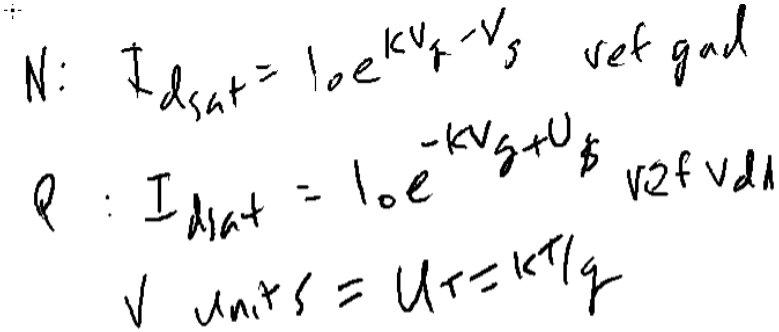
1. Draw four-terminal symbols for native and well transistors and label all the terminals; use *d* for drain, *s* for source, *g* for gate, *b* for bulk, and *w* for well. Indicate the direction of current flow that is consistent with your choice of drain and source (you can drag the * .png file into the cell below).



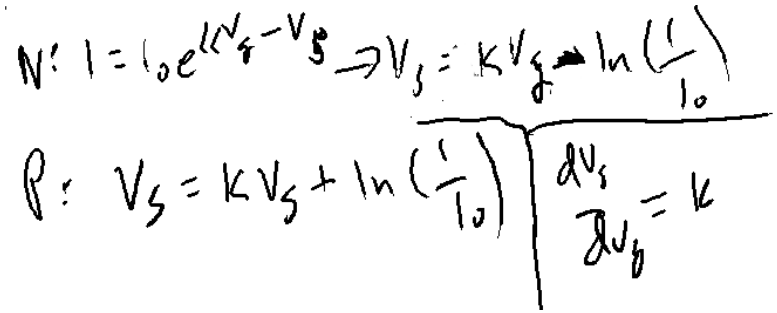
2. Write the expressions for the subthreshold (weak inversion) current I_{ds} for both types of transistors.



3. Write the expressions for the *saturation* current of these transistors, that is, the value of the current when $|V_{ds}| \gg \frac{4kT}{q}$. For the remaining questions you may assume that the transistor is in saturation.



4. For both transistors, write an expression for source voltage as a function of gate voltage if the channel current is constant and the transistor is in saturation. In each case, what is $\frac{\mathrm{d}V_s}{\mathrm{d}V_g}$?



ESD protection of CMOS Chips (need to know)

All MOSFET chips are *extremely* prone to damage by static electricity. The current through the transistors is controlled by an insulated gate.

Not so fun facts: **Even a few tens of volts can blow up the gate. A short walk across the room can build up kilovolts of static potential.**

There are electrostatic discharge (ESD) protection structures on the chip inputs that are designed to leak off the static charge before it can damage the chip, but often this will not be enough.

There is one simple precautions that can definitely keep the chip safe.

When the PCB is not powered up always ground yourself to your computer ground (e.g. the outer shell of your USB connector) before picking up or touching the board. This will discharge the static charge. Of course this assumes your computer is plugged into the wall power with a grounded power cable. In general, try to make sure your body charge is discharged before you handle the PCB.

Experiments and Lab Reports

For the following experiments, include in your lab reports, graphs of all theoretical and experimental curves. Experimental data should be plotted in a point style so that individual data points are visible. Make sure you take enough data points and label your axes. The theoretical fit should be graphed on the same plot in a line style.

Your written interpretation of the results and any anomalies are essential. Please do not just hand in the plots without any interpretation on your part. Your report does not need to be beautiful, but it should show that you understand what you are measuring.

Remember that the purpose of this lab is to investigate *subthreshold* transistor characteristics. Therefore, all voltage sweeps should span the measurable subthreshold regime while extending just far enough above threshold to show where the threshold is.

Avoid these common mistakes in your report:

- **Not discussing your data sufficiently.** Think about a publication. The readers want to understand your reasoning with you. They want to be able to reproduce your results.
- **Not using cross-hair axes when 0,0 is relevant.** Use grid on to turn on grid, which will draw dotted lines from major ticks. See fontsize to make spacing readable.
- **Forgetting to mention what your plot shows.**
- **Not labeling your figures with a caption,** e.g., "Fig. 1: Transistor drain current vs. gate voltage, Experiment 1."
- **Insufficiently annotating your data.** It's OK to draw on your plots to indicate the slope of the curve, or the x / y intercepts.
- **Using identical markers for all plots.** Your curves must be distinguishable when printed in black and white. Use e.g. plot(v,i,'o-',v,i2,'s-'), which labels one curve with circle markers and the other with square markers.
- **Forgetting units on measurements,** e.g. "our conductance is 1.000653e-10". What are the units?
- **Giving your measurements too many digits of precision;** see previous error. Do your instruments really give you 7 digits of precision?

2 Set up the experiment

Load all the necessary libraries for this notebook, and define a 'datapath' variable to hold the path to your precious data that you will store and maybe load later

In [ ]:
```
# import the necessary library to communicate with the hardware
import pyplane # to talk to CoACH board
import time # for time.sleep(seconds)
import numpy as np # numpy for arrays etc
from scipy import stats # for stats.linregress
import matplotlib.pyplot as plt # for plotting
import matplotlib
plt.rcParams.update({'font.size': 14}) # make the detault font size large
matplotlib.rcParams['pdf.fonttype'] = 42 # save fonts as type that are no
from engineering_notation import EngNumber as ef # format numbers in engi
from pathlib import Path # used for saving data
import tqdm # great for showing progress in loops
from jupyter_save_load_vars import savevars,loadvars # to save and load y

datapath = Path('data/lab2') # make a data folder to save your data calle
datapath.mkdir(parents=True, exist_ok=True)

# below lines are notebook magic for debugging, you can uncomment them wh
# %load_ext autoreload
# %autoreload explicit
# %aimport ne1
from ne1 import * # savevars(file) saves your variables, loadvars() loads
```

Saving and loading data

Use the blocks below to save and load your data

Saving data

In [ ]:
```
# you probably don't want this the first time you work on lab... use it a
# savevars(datapath/'lab2')
```

Loading previous data

In [ ]:
```
# loadvars(datapath/'lab2',overwrite='yes')
# loadvars(datapath/'lab2') # use to prompt each variable overwrite
```

Connect the device

In [ ]:
```
p=Coach() # make the coach object to do measurements
p.open() # you might need to give a port name here, e.g. '/dev/ttyACM1'.

# Note that if you plug out and plug in the USB device in a short time in
# then you may get error messages with open(...ttyACM0). So please avoid
```

```
[INFO]: 2023-10-12 13:05:09,599 - NE1 - Opened CoACH at /dev/ttyACM0 with
firmware version (1, 12, 5) (File "/root/GitLab/CoACH-labs/ne1.py", line 1
39, in open)
```

If you get any error, e.g. `attempting to stat /dev/ttyACM0: No such file or directory`, then make sure the CoACH board is connected (and maybe mapped through the virtual machine USB).

Troubleshooting: See troubleshooting section at end of readme.md and Coach_Teensy_interface

In [ ]:
```
p.get_firmware_version() # you can use this cell to check if the board is
```

Out[ ]:
```
(1, 12, 5)
```

In [ ]:
```
#use this cell to close the board and free resources
p.close()
```

```
[INFO]: 2023-10-12 13:05:09,618 - NE1 - closing device (deleting pyplane.P
lane() object (File "/root/GitLab/CoACH-labs/ne1.py", line 202, in close)
```

In [ ]:
```
# use this cell to try different types of resets
# p.reset_soft()
# p.reset_hard()
```

In [ ]:
```
# use this code block to reproduce timeout bug,
# import time
# while True:
#     p.read_current(pyplane.AdcChannel.GO0_N)
#     time.sleep(.01)
```

Recommendations to this lab

- You do not need to follow the order, it is actually better to do all the measurement of one device together, e.g. 3.1 -> 4.1 -> 3.2 -> 4.2
- **Please save the data as frequently as possible and use the loaded data for processing**

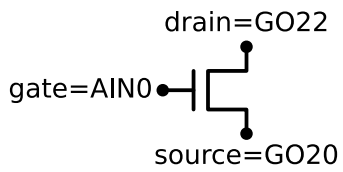
3 Current as a Function of Gate Voltage

3.1 N-FET saturation current I_{ds} vs V_{gs}

For the N-FET device on the CoACH chip, measure current I_{ds} as a function of gate voltage V_g for fixed source, bulk (substrate or well), and drain voltages. This experiment replicates the one you first did in lab1.

In [ ]:
```
# uses schemdraw, you may have to install it in order to run it on your P
import schemdraw
import schemdraw.elements as elm
d = schemdraw.Drawing()
Q = d.add(elm.NFet, reverse=True)
d.add(elm.Dot, xy=Q.gate, lftlabel='gate=AIN0')
d.add(elm.Dot, xy=Q.drain, toplabel='drain=GO22')
d.add(elm.Dot, xy=Q.source, botlabel='source=GO20')
d.draw()
```

Out[ ]:



- To select the NFET you will measure, you have to set the input voltage and output current demultiplexer by sending a configuration event :

In [ ]:
```
# configure muxes to measure NFET
p.setup_nfet()
```

```
[INFO]: 2023-10-12 13:05:09,777 - NE1 - Opened CoACH at /dev/ttyACM0 with
firmware version (1, 12, 5) (File "/root/GitLab/CoACH-labs/ne1.py", line 1
39, in open)
```

- What will be the fixed value for source, bulk (substrate or well), and drain voltages?
- We give you some values that don't make sense. **Correct the values below and set them to put the NFET in subthreshold current mode in saturation**.

In [ ]:
```
# set source voltage
vs_n = 1
v=p.set_nfet_vs(vs_n)
print(f"The source voltage is set to {v:.3f} V")
```

```
The source voltage is set to 0.999 V
```

In [ ]:
```
# set drain voltage
vd_n = 1  # NOTE fixed Vd=1V (was zero!)
v=p.set_nfet_vd(vd_n)
print(f"The drain voltage is set to {v:.3f} V")
```

```
The drain voltage is set to 0.999 V
```

In [ ]:
```
# set trial gate voltage
vg_n = 1.8 # NOTE fixed Vg=.5 for subthreshold (was Vdd)
v=p.set_nfet_vg(vg_n)
print(f"The trial gate voltage is set to {v:.3f} V")
```

```
The trial gate voltage is set to 1.798 V
```

In [ ]:
```
# read Ids, from *Source* --> NOTE THAT THE ADC CHANNEL PIN CHANGES THE N
ids_n = p.measure_nfet_id()
print(f"Ids is {ef(ids_n)}A") # Note how ef() function formats engineerin
```

```
Ids is 39.79nA
```

Note that we corrected the voltages to produce a subthreshold current

Data acquisition for I_{ds} vs V_{gs} for NFET

Sweep the gate voltage over relevant exponential range of current and plot the data you collect on log scale for current, as you did in lab1.

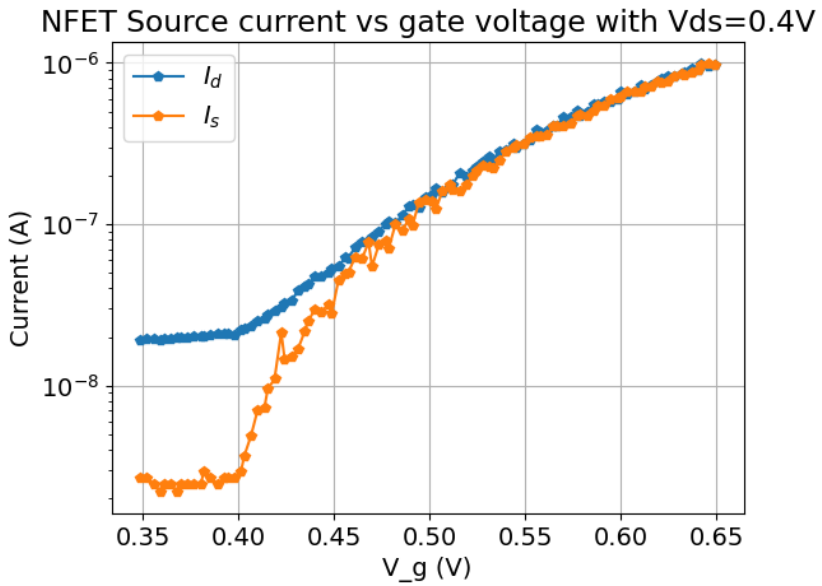
Here is some code updated from lab1.

Note how we measure both source I_s and drain I_d currents to see which looks better for analysis.

In [ ]:
```
p=Coach()
p.setup_nfet()
# set source voltage
vs_n = 0
v=p.set_nfet_vs(vs_n)
print(f"The source voltage is set to {vs_n:.3f}V")
# set drain voltage
vd_n = .4
v=p.set_nfet_vd(vd_n)
print(f"The drain voltage is set to {vd_n:.3f}V")
# set starting gate voltage
vg_n = 0
v=p.set_nfet_vg(vg_n)
print(f"The starting gate voltage is set to {vg_n:.3f}V")

# make a V_g array for the gate voltages ranging e.g. from 0.4V to 0.8V.
# you can adjust this range to cover the realistic measurement range.
n_samples=100 # now many points to capture. Note the DAC has only 10 bits
vgn_low=.35  # TODO set this line and next line to grab just the range yo
```

```
vgn_high=.65 # NOTE fixed high end to match range
vgn_set=np.linspace(vgn_low,vgn_high,n_samples) # make linear spacing wit
vgn_actual=np.zeros_like(vgn_set) #we will use this to hold the actual DA
#Initialize current variables
idn_meas=np.zeros(n_samples) # drain currents
isn_meas=np.zeros(n_samples) # source currents
import time
#Read Ids at Vg sweep and wait for it to settle
delay=0.01
for n in range(n_samples):
    vgn_actual[n]=p.set_nfet_vg(vgn_set[n]) # record the voltage after DA
    time.sleep(delay) # sleep a bit for FET to settle
    idn_meas[n]=p.measure_nfet_id()
    isn_meas[n]=p.measure_nfet_is()
    if n%10==0:
        print(f'Vg={vgn_actual[n]:8.3f}V idn_meas={ef(idn_meas[n])}A')
p.close()

#Plot in log scale
plt.semilogy(vgn_actual, idn_meas,'-p')  #plot using line with points mar
plt.semilogy(vgn_actual, isn_meas,'-p')  #plot using line with points mar
plt.legend(['$I_d$','$I_s$'])
plt.xlabel('V_g (V)')
plt.ylabel('Current (A)')
plt.title(f'NFET Source current vs gate voltage with Vds={vd_n-vs_n}V')
plt.grid(True) # turn on grid
import os
os.makedirs('data',exist_ok=True)
plt.savefig(datapath/'lab2-nfet-ids-vs-vgs.pdf') # change the name to wha
plt.show() # render the plot, you need to show the plot *after* you save
```

```
[INFO]: 2023-10-12 13:22:26,345 - NE1 - Opened CoACH at /dev/ttyACM0 with
firmware version (1, 12, 5) (File "/root/GitLab/CoACH-labs/ne1.py", line 1
39, in open)
The source voltage is set to 0.000V
The drain voltage is set to 0.400V
The starting gate voltage is set to 0.000V
With Vg=0, the offset current I0n=18.80nA
Vg=   0.348V idn_meas=19.29nA
Vg=   0.380V idn_meas=20.02nA
Vg=   0.410V idn_meas=24.90nA
Vg=   0.440V idn_meas=47.12nA
Vg=   0.470V idn_meas=83.01nA
Vg=   0.501V idn_meas=152.83nA
Vg=   0.531V idn_meas=261.72nA
Vg=   0.561V idn_meas=375.49nA
Vg=   0.591V idn_meas=570.56nA
Vg=   0.621V idn_meas=803.47nA
[INFO]: 2023-10-12 13:22:27,732 - NE1 - closing device (deleting pyplane.P
lane() object (File "/root/GitLab/CoACH-labs/ne1.py", line 202, in close)
```

Plotting drain current vs gate voltage on log scale

Which measurement (drain or source) looks better to analyze? Why do you think this is occurring?

Clearly the source current is better. The drain has some huge leakage, probably to ground from the high drain voltage which is set here to 400mV

If your data looks good, maybe you want to save it? You can ignore the error about not being able to pickle p; it is a native code object.

In [ ]:
```
# savevars(datapath/'lab2')
```

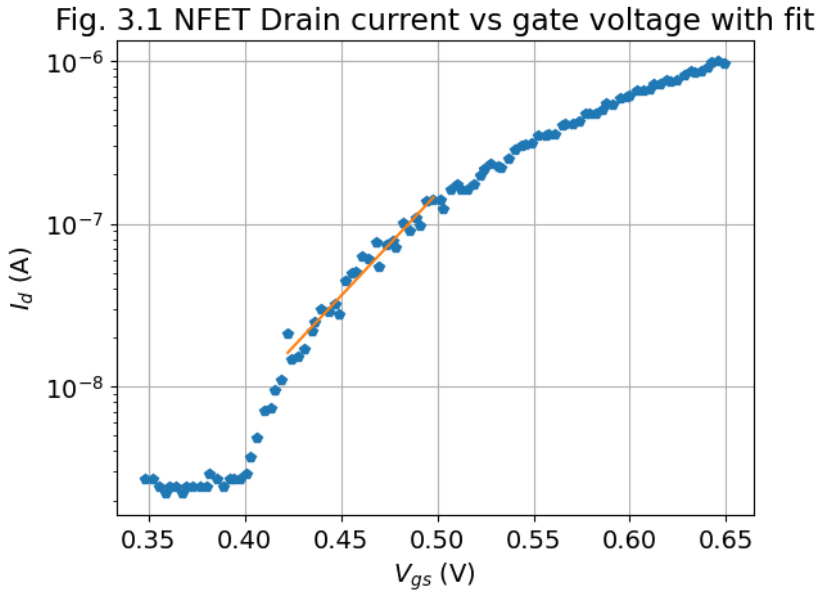
Fitting an exponential to the NFET data

Now you can fit a line to the valid region in log plot and try to extract I_0 and κ.

Unfortunately there is no decent interactive line fitting tool for python notebooks. You need do do a lot of ugly slicing and filtering to fit just the range you want. Please check the code below; you need to adapt it for the PFET drain curve fits later.

In [ ]:
```
# NOTE we corrected the measurement to source current

idn_corrected=isn_meas # compute the current with no offset subtracted

# idn_corrected=idn_meas # compute the current with no offset subtracted
# idn_corrected=Id_meas-Idn0 # compute the current with offset subtracted

# note that this correction could make estimate of i0n (the off current)
# or could you still estimate it from the intercept of the fit to the cur
```

```
# fit in the valid range (you may want to add the fitted line in the plot
# hint: use scipy.linregress https://docs.scipy.org/doc/scipy/reference/g
# take only the valid range for fitting by finding the indices of Vg betw
# we also need to ignore Id values <0 for log fit
idix=np.where(idn_corrected>0) # find indices of currents >0 so log makes
idlog=np.log(idn_corrected[idix]) # compute natural log of current to fit
vg_masked=vgn_actual[idix]
vg0=.42  # NOTE changed range to smaller currents for more reasonable kap
vg1=.5
idx=np.where((vg_masked>vg0) & (vg_masked<vg1)) # find indices that are i
x=vg_masked[idx] # get the vg values
y=idlog[idx] # and id values
fit=stats.linregress(x,y) # fit a line to them; see https://docs.scipy.or
idn_fit=np.exp(fit.intercept+x*fit.slope)
i0n=np.exp(fit.intercept) # compute current at Vg=0
# fit.slope is efolds/volt
U_T=25e-3 # thermal voltage in Volts
q=1.6e-19 # elementary change in Coulombs
v_per_efold=1/fit.slope # compute the volts for one "e-fold" (factor of i
# compute kappa from UT and v_per_efold
kappa=U_T/v_per_efold

#Plot in log scale
plt.semilogy(vgn_actual, idn_corrected,'p',label='measured currents')  #p
plt.semilogy(x, idn_fit,'-',label='fit')  # plot as line
plt.xlabel('$V_{gs}$ (V)')
plt.ylabel('$I_d$ (A)')
plt.title('Fig. 3.1 NFET Drain current vs gate voltage with fit')
plt.grid(True) # turn on grid
import os
os.makedirs('data',exist_ok=True)
plt.savefig(datapath/'lab2-nfet-ids-vs-vgs-fitted.pdf') # change the name
plt.show() # render the plot, you need to show the plot *after* you save
print(f'Exponential fit between {vg0}V and {vg1}V has i0n={ef(i0n)}A ({ef


kappa_n=kappa # save data for later
v_per_efold_n=v_per_efold
```


```
Exponential fit between 0.42V and 0.5V has i0n=76.56fA (478.49ke/s) and in
verse slope 34.45mV/e-fold; kappa=0.726 assuming U_T=25mV
```

Please comment on your results.

1. Over what current range does your exponential fit make sense, considering the limitations of the current sensing?
2. Does the value of the off current I_0 make sense?
3. How many mV must V_g change to change the current by a factor of e?
4. Does the value of κ make sense?

Answers: Original measurement of drain current had excessive leakage and the kappa value of 0.4 is much too small. Changed to measure smaller currents spanning 10nA to 100nA.

1. Value of κ of about 0.6 is more reasonable for real value.
2. Value of $I_0 = 0.5pA$ seems quite small considering that the threshold voltage V_T is only about 0.55V. We don't trust this result too much.
3. 40mV/e-fold on gate is a reasonable number for most standard FETs.
4. Yes, with change in measurements.

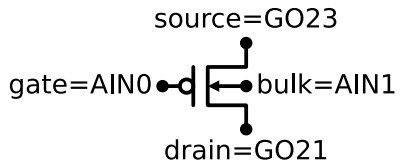
In summary we cannot trust this measurements very much because this behavior is too non-physical for subthreshold.

3.2 P-FET saturation current I_{ds} vs V_{gs}

Now you should replicate above experiment but using the test PFET

In [ ]:
```
# uses schemdraw, you may have to install it in order to run it on your P
import schemdraw
import schemdraw.elements as elm
d = schemdraw.Drawing()
Q = d.add(elm.PFet, reverse=True, bulk=True)
d.add(elm.Dot, xy=Q.gate, lftlabel='gate=AIN0')
d.add(elm.Dot, xy=Q.bulk, rgtlabel='bulk=AIN1')
d.add(elm.Dot, xy=Q.drain, botlabel='drain=GO21')
d.add(elm.Dot, xy=Q.source, toplabel='source=GO23')
d.draw()
```

Out[ ]:



- What will be the fixed source, bulk (substrate or well), and drain voltages for the PFET?

In [ ]:
```
# configure muxes to measure PFET
p=Coach()
p.open()
p.setup_pfet()

# set bulk voltage; HINT it must be the positive supply voltage 1.8V
vdd=1.8 # power supply voltage in V
vb_p = vdd
p.set_pfet_vb(vb_p)
print(f"The bulk voltage is set to {vb_p}V")
# set source voltage; HINT it must be the same as the nwell bulk voltage
vs_p = vdd
p.set_pfet_vs(vs_p)
print(f"The source voltage is set to {vs_p}V")
# set drain voltage; HINT it should be a lower voltage, can be ground (0)
vd_p = vdd
p.set_pfet_vd(vd_p)
print(f"The drain voltage is set to {vd_p}V")
# set trial gate voltage
vg_p = vdd
p.set_pfet_vg(vg_p)
print(f"The gate voltage is set to {vg_p}V")
# read Ids
is_p = p.measure_pfet_is() # note here we measure the PFET current at its
id_p = p.measure_pfet_id() # note here we measure the PFET current at its
print(f"Id={ef(id_p)}A, Is={ef(is_p)}A") # note again how we use f-string
```

```
[INFO]: 2023-10-16 12:57:38,023 - NE1 - closing device (deleting pyplane.P
lane() object (File "/home/tobi/Dropbox/GitLab/CoACH-labs/ne1.py", line 20
7, in close)
[INFO]: 2023-10-16 12:57:38,053 - NE1 - Found CoaACH at /dev/ttyACM1 (File
"/home/tobi/Dropbox/GitLab/CoACH-labs/ne1.py", line 184, in find_coach)
[INFO]: 2023-10-16 12:57:38,055 - NE1 - Opened CoACH at /dev/ttyACM1 with
firmware version (1, 12, 5) (File "/home/tobi/Dropbox/GitLab/CoACH-labs/ne
1.py", line 144, in open)
```



```
The bulk voltage is set to 1.8V
The source voltage is set to 1.8V
The drain voltage is set to 1.8V
The gate voltage is set to 1.8V
Id=5.83uA, Is=6.40uA
```

With vg_p set to $V_{dd} = 1.8V$ you should get a small current of a few nA. With vg_p set to ground (0 volts), you should get a saturating current of >1uA.

You can use this little code block to make sure the PFET is making sense, and to find a reasonable range over which to sweep vg_p.

- Data acquisition and plotting for PFET; repeat the NFET measurements here but for PFET

You can copy and adapt the code from above

In [ ]:
```
p.setup_pfet()
# set source voltage
vdd=1.8 # power supply voltage in V
vb_p = vdd
vs_p = vdd
vd_p = 0 # NOTE that it needn't be at ground, just enough to saturate the
p.set_pfet_vb(vb_p)
p.set_pfet_vs(vs_p)
p.set_pfet_vd(vd_p)

# make a V_g array for the gate voltages ranging e.g. from 0.4V to 0.8V.
# you can adjust this range to cover the realistic measurement range.
n_samples=100
vg_high=1.5 # starting gate voltage
vg_low=1 # ending gate voltage
vgp_set=np.linspace(vg_high,vg_low,n_samples) # make linear spacing with
vgp_actual=np.zeros_like(vgp_set) # we will use this to hold the actual D
#Initialize current variables
isp_meas=np.zeros(n_samples) # source current
idp_meas=np.zeros(n_samples) # drain current
import time
#Read Ids at Vg sweep and wait for it to settle
delay=0.01
for n in range(n_samples):
    vgp_actual[n]=p.set_pfet_vg(vgp_set[n])
    time.sleep(delay) # delay to allow device to settle
    isp_meas[n]=p.measure_pfet_is()
    idp_meas[n]=p.measure_pfet_id()
    if n%20==0:
        print(f'Vg={vgp_actual[n]:8.3f}V Is_meas={ef(isp_meas[n])}A')
```

```
With Vgs=0, the offset current Id0=83.01nA
Vg=   1.499V Is_meas=6.10nA
Vg=   1.399V Is_meas=6.59nA
Vg=   1.297V Is_meas=80.57nA
Vg=   1.196V Is_meas=323.97nA
Vg=   1.094V Is_meas=829.83nA
```

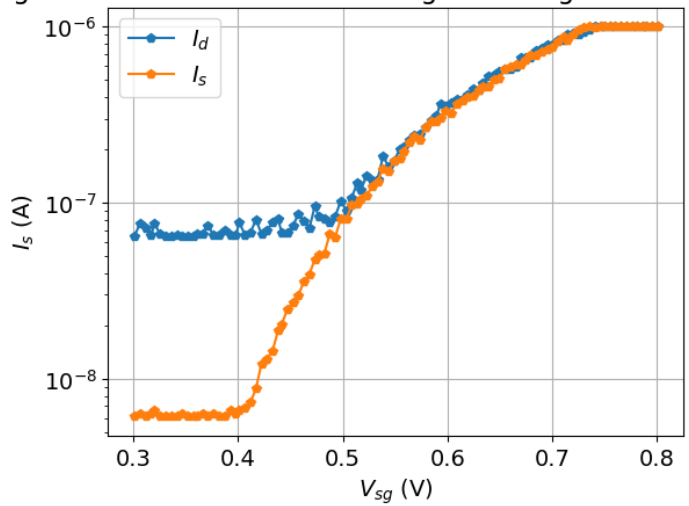
In [ ]:
```
# if something goes wrong use this cell before unplugging board
p.close()
```

```
[INFO]: 2023-10-11 18:15:02,030 - NE1 - closing device (deleting pyplane.P
lane() object (File "/root/GitLab/CoACH-labs/ne1.py", line 202, in close)
```

Now plot the PFET data. Note we use $V_{sg} - V_s - V_g$ for x-axis because the source is at a higher voltage.

In [ ]:
```
#Plot data in log scale
vsgp=vdd-vgp_actual # compute the gate voltage from Vdd for plotting more
plt.semilogy(vsgp, idp_meas,'-p')
plt.semilogy(vsgp, isp_meas,'-p')
plt.legend(['$I_d$','$I_s$'])

plt.xlabel('$V_{sg}$ (V)')
plt.ylabel('$I_{s}$ (A)')
plt.title(f'Fig. 3.2 PFET Source current vs gate voltage with Vsd={vs_p-v
plt.grid(True) # turn on grid
plt.show()

# Id_corrected=Id_meas-Id0 # compute the current with offset subtracted
# note that this correction could make estimate of I_0 (the off current)
# or could you still estimate it from the intercept of the fit to the cur
```

Fig. 3.2 PFET Source current vs gate voltage with Vsd=1.8V



In [ ]:
```
# if the data looks good it might be good to save it too
savevars(datapath/'lab2')
```



```
[INFO]: 2023-10-11 18:15:02,434 - saveloadvars - saved to data/lab2/lab2.d
ill variables [ datapath p vs_n v vd_n vg_n ids_n I0n n_samples vgn_low vg
n_high vgn_set vgn_actual idn_meas isn_meas delay n idn_corrected idlog vg
_masked vg0 vg1 x y fit idn_fit i0n U_T q v_per_efold kappa vdd vb_p vs_p
vd_p vg_p ids_p I0p vg_high vg_low vgp_set vgp_actual isp_meas idp_meas vs
gp isp_corrected vsgp_masked idp_fit i0p v_per_efold_p pre kappa_n v_per_e
fold_n ] (File "/root/miniconda3/envs/ne1/lib/python3.8/site-packages/jupy
ter_save_load_vars.py", line 179, in savevars)
[WARNING]: 2023-10-11 18:15:02,434 - saveloadvars - could not pickle:
['d', 'Q', 'idix', 'idx'] (File "/root/miniconda3/envs/ne1/lib/python3.8/s
ite-packages/jupyter_save_load_vars.py", line 181, in savevars)
```

Commment: the measurements of this PFET don't make much sense. They should NOT curve like this. We strongly suspect some problem with current ADC leakage. But clearly the source end is better to measure even though it is at Vdd. The leakage on it should be fixed if it is leaking to ground. Still, the curvature is not sensible. Fits to this data are highly suspect.

Fitting an exponential to the PFET data

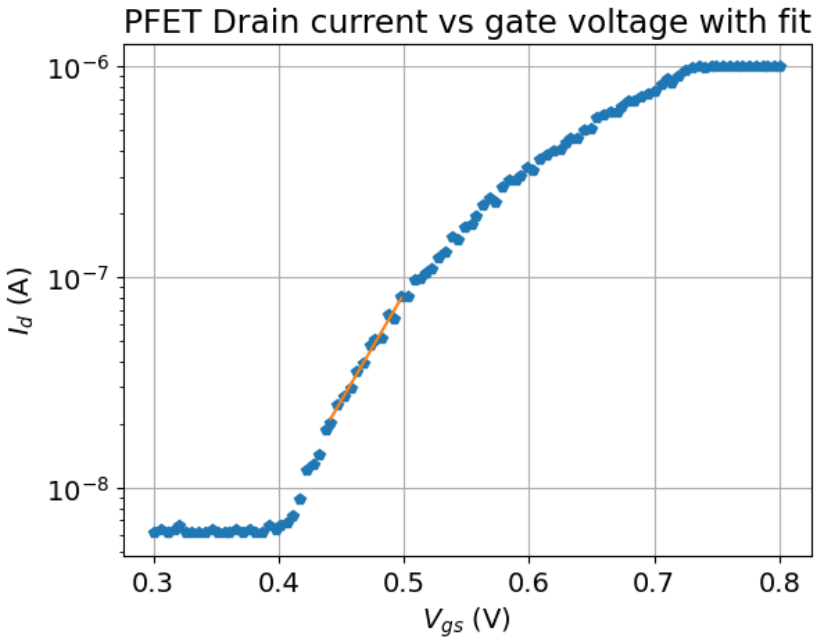
Now you can fit a line to the valid region in log plot and try to extract I_0 and κ.

We have adapted the NFET code for you; you can just adjust it for your data.

- Can you improve the fit and curve by subtracting I_{leak}, the instrumentation leakage current?

In [ ]:
```
isp_corrected=isp_meas # compute the current with no offset subtracted,

# note that this correction could make estimate of i0n (the off current)
# or could you still estimate it from the intercept of the fit to the cur

# fit in the valid range (you may want to add the fitted line in the plot
# hint: use scipy.linregress https://docs.scipy.org/doc/scipy/reference/g
# take only the valid range for fitting by finding the indices of Vg betw
# we also need to ignore Id values <0 for log fit
idix=np.where(isp_corrected>0) # find indices of currents >0 so log makes
idlog=np.log(isp_corrected[idix]) # compute natural log of current to fit
vsgp_masked=vsgp[idix]
vg0=.44  # set limits for fit
vg1=.5  # NOTE changed for steeper range
idx=np.where((vsgp_masked>vg0) & (vsgp_masked<vg1)) # find indices that a
x=vsgp_masked[idx] # get the vg values
y=idlog[idx] # and id values
fit=stats.linregress(x,y) # fit a line to them; see https://docs.scipy.or
idp_fit=np.exp(fit.intercept+x*fit.slope) # compute the fitted values on
i0p=np.exp(fit.intercept) # compute current at Vg=0
# fit.slope is efolds/volt
U_T=25e-3 # thermal voltage in Volts
q=1.6e-19 # elementary change in Coulombs
v_per_efold=1/fit.slope # compute the volts for one "e-fold" (factor of i
# compute kappa from UT and v_per_efold
kappa=U_T/v_per_efold

#Plot in log scale
plt.semilogy(vsgp, isp_corrected,'p',label='measured currents')  #plot as
plt.semilogy(x, idp_fit,'-',label='fit')  # plot as line
```

```
plt.xlabel('$V_{gs}$ (V)')
plt.ylabel('$I_d$ (A)')
plt.title('PFET Drain current vs gate voltage with fit')
plt.grid(True) # turn on grid
import os
os.makedirs('data',exist_ok=True)
plt.savefig(datapath/'lab2-pfet-ids-vs-vgs-fitted.pdf') # change the name
plt.show() # render the plot, you need to show the plot *after* you save
print(f'Exponential fit between {vg0}V and {vg1}V has i0p={ef(i0p)}A ({ef

kappa_p=kappa # save data for later
v_per_efold_p=v_per_efold
```




```
Exponential fit between 0.44V and 0.5V has i0p=696.39fA (4.35Me/s) and inv
erse slope 34.45mV/e-fold; kappa=0.585 assuming U_T=25mV
```

Comment: This original measurement was not very sensible considering the wierd behavior of the PFET. A κ of 0.2 is ridiculous - no one would buy these lousy FETs.

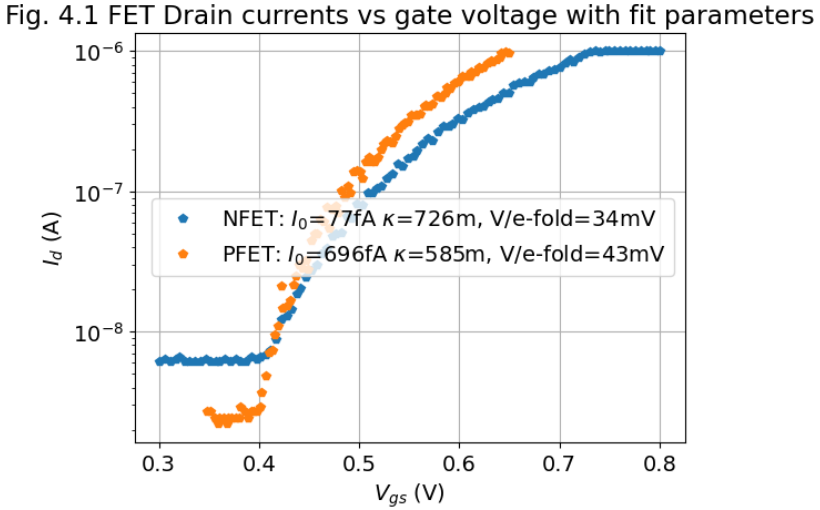
We changed the range to a more sensible one. The kappa of 0.7 is more in the expected range, but we are not satisfied that the cause of this big nonideality is not understood.

4. Final summary plot of subthreshold transconductance behavior

- Plot NFET and PFET offset-corrected data together in one plot
- Use plt.legend with f=strings to annotate the sumamry I_0 mV/e-fold and κ values for each FET type

In [ ]:
```
#Plot in log scale
plt.semilogy(vsgp, isp_corrected,'p',label='measured currents')  #plot as
plt.semilogy(vgn_actual, idn_corrected,'p',label='measured currents')  #p

plt.xlabel('$V_{gs}$ (V)')
plt.ylabel('$I_d$ (A)')
plt.title('Fig. 4.1 FET Drain currents vs gate voltage with fit parameter
plt.grid(True) # turn on grid


pre=0 # digits precision for labels
nfet_label=f'NFET: $I_0$={ef(i0n,precision=pre)}A $\kappa$={ef(kappa_n,pr
pfet_label=f'PFET: $I_0$={ef(i0p,precision=pre)}A $\kappa$={ef(kappa_p,pr
plt.legend([nfet_label,pfet_label])
```

Out[ ]:
```
<matplotlib.legend.Legend at 0x7fec406ffc70>
```



- Include here a summary of your final data and what the results mean

Summary: The final data is not so great but reasonable. At least we see exponential behavior over a limited range and managed to extra parameters that are not too wildly incorrect. However, the instrumentation leaves a lot to be desired.

Postlab

The answers to the postlab should be included below.

Today, most processes are double-well or *twin-tub* both n- and p-wells are implanted in a lightly doped epitaxially-grown p-substrate.

Epitaxially means that the layer is grown atom by atom by chemical vapor deposition, resulting in a very regular and pure crystal structure.

The classchips you are using were fabricated in a 180nm process. For an *n*-well device, the *n*-type material of the well is the bulk, while the active areas (source and drain) are *p*-type. The gate voltage must force all the electrons in the *n*-well away from the surface. The resulting depletion region provides a channel for holes through enemy territory (n-well) separating the p-type source and drain. If the bulk is heavily doped, the gate must work harder to repel electrons.

Another way of saying this is that the capacitance of this depletion layer and the gate oxide capacitance form a *capacitive divider* that determines how much of the gate voltage appears at the surface channel. If the depletion layer is thin, the depletion capacitance will be large and hence the divider ratio will be unfavorable.

(1) How does the thickness of the depletion region depend on the doping density and on the channel (surface) potential? Assume that the doping density is uniform.

Answer: Higher doping gives thinner depletion. Higher ψ_s increases depletion region width.

(2) Explain why κ varies with the source voltage at constant current (as in the source follower).

Answer: As V_s increases, V_g and ψ_s also must increase to maintain constant $\kappa V_g - V_s$. At higher ψ_s, we get wider depletion region under the channel, which decreases C_{dep}, which increases κ.

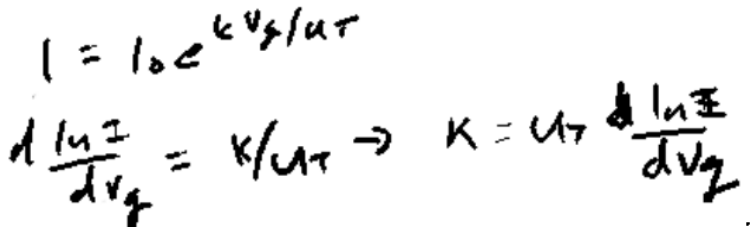
(3) Is there a difference in your measured κ between the n- and p-type devices? Explain the possible reason(s).

Answer: The results are too unreliable to conclude anything definitive. But if we could trust the κ values, then they would mean that the higher κ value of the NFET means that the pwells are more lightly doped than the nwells.

(4) From your results in the experiments, state under what conditions the assumption that κ is constant is reasonable.

Hint: You can plot the derivative of $\log(I_{ds})$ vs V_{gs}; how is this slope related to the instantaneous κ value?

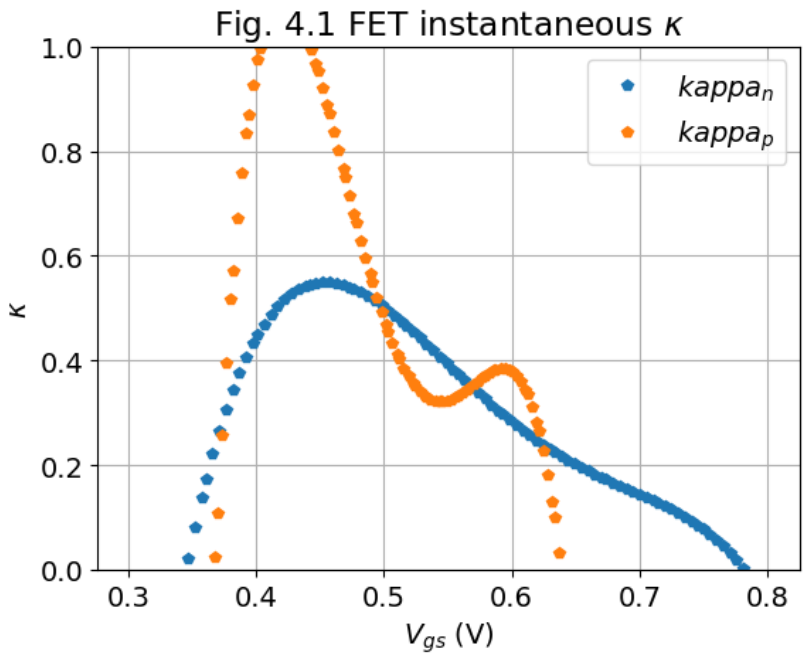
Answer:



We used a 5th order spline to smooth the data before computing the local κ. Even so, the results are not clear.

In [ ]:
```
# compute the instantaneous kappa from d(log(I))/d(V)
# computing slope from raw data is extremely noisy. We use spline here to
from scipy.interpolate import splrep, splev

ut=25e-3
log_in_smooth=np.log(idn_corrected)
n_spline=splrep(vgn_actual,log_in_smooth,k=5,s=3)
k_n=ut*np.gradient(splev(vgn_actual,n_spline),vgn_actual)

log_ip_smooth=np.log(isp_corrected)
p_spline=splrep(vsgp,log_ip_smooth,k=5,s=3)
k_p=ut*np.gradient(splev(vsgp,p_spline),vsgp)

#Plot in log scale
plt.plot(vsgp, k_p,'p')  #plot as points
plt.plot(vgn_actual, k_n,'p')  #plot as points
# clip to 0-1 range
ax = plt.gca()
ax.set_ylim([0, 1])

plt.xlabel('$V_{gs}$ (V)')
plt.ylabel('$\kappa$')
plt.title('Fig. 4.1 FET instantaneous $\kappa$')
plt.grid(True) # turn on grid
plt.legend(['$kappa_n$','$kappa_p$'])
```

Out[ ]:
```
<matplotlib.legend.Legend at 0x7fec3b609f10>
```



Answer: There is not so much we can really conclude from this data. The I_{ds} measurements are just too bad. The expected behavior is that κ should increase slightly as we approach threshold.